

Components of IoT Application

Gleb Bulygin

gbulygin@students.oamk.fi

DIN24SP

Autumn 2026

Week 7

58. Describe shortly following security tools/terms/concepts:

- **CVE**

Common Vulnerabilities and Exposures. A public list where every known vulnerability gets its own ID, like `CVE-2021-44228` (Log4Shell). It's run by MITRE, so everyone (vendors, scanners, news) can refer to the same bug by the same name.

- **CVSS**

Common Vulnerability Scoring System. Gives a vulnerability a severity score from 0.0 to 10.0, based on things like how it can be reached (network or local), how hard it is to exploit, whether it needs a login or user action, and what it affects (confidentiality, integrity, availability). The score bands are Low, Medium, High (7.0–8.9) and Critical (9.0–10.0).

- **Asymmetric encryption**

Uses a key pair: a **public key** that anyone can have and a **private key** that only the owner keeps. Data encrypted with the public key can only be decrypted with the private key, and signing works the other way round. It's slow, so it's mostly used for key exchange and digital signatures. Examples: RSA, ECC.

- **Symmetric encryption**

The same secret key is used to encrypt and decrypt. It's fast and used for the actual data, but both sides need to get the key somehow, which is usually done with asymmetric encryption first. Examples: AES, ChaCha20.

- **Disassembler**

A tool that turns compiled machine code back into assembly instructions, so you can see what a program does without its source code. Used in reverse engineering and malware analysis. Examples: objdump, IDA, Ghidra.

- **Overflow vulnerability**

A program writes more data into a buffer than it has room for, and the extra data overwrites memory next to it. On the stack this can overwrite the return address and let an attacker run their own code. It's typical of C/C++ code that doesn't check input lengths. Integer overflows are a related bug where a number wraps around and leads to wrong size calculations.

- **Race condition vulnerability**

The result depends on the timing of two things happening at the same time. A classic case is TOCTOU (time-of-check to time-of-use): a program checks that a file is safe, and before it opens it, the attacker swaps the file for a link to something else.

- **ASLR/DEP/NX**

Memory protections that make overflow exploits harder:

- **ASLR** (Address Space Layout Randomization) puts the stack, heap and libraries at random addresses each run, so the attacker can't know where to jump.
- **DEP** (Data Execution Prevention, Windows name) and the **NX** bit (No-eXecute, the CPU feature behind it) mark data areas like the stack as non-executable, so injected code can't run there.

- **Ghidra**

A free, open-source reverse engineering tool released by the NSA in 2019. It disassembles and also **decompiles** binaries into C-like code, supports many CPU architectures (useful for IoT firmware), and can be scripted.

- **RCE vulnerability**

Remote Code Execution. An attacker can run their own commands or code on the target over the network. It's usually the most serious type, because it often means full control of the device or server.

- **Local privilege escalation**

An attacker who already has a normal user account on a system finds a bug or misconfiguration that gives them higher rights, usually root or Administrator. Examples are kernel bugs, badly configured `sudo` rules or SUID programs.

- **Zero-day vulnerability**

A vulnerability that the vendor doesn't know about yet, or hasn't fixed yet, so defenders have had "zero days" to patch it. Zero-days used in real attacks are very valuable and hard to defend against.

- **Zero-click exploit**

An exploit that needs no action from the victim, so no link to click and no file to open. Just receiving a message or a network packet is enough. Example: the Pegasus spyware hacked iPhones through iMessage this way.

- **SQL injection**

User input is put straight into a database query, so the attacker can change the query. For example, entering `' OR '1'='1` as a password can skip the login check, and other inputs can read or delete data. It's prevented by using parameterized queries (prepared statements).

- **Command injection vulnerability**

Like SQL injection, but the input ends up in an operating system command. If a web page runs `ping <user input>`, entering `8.8.8.8; cat /etc/passwd` makes the server run the second command too. It's very common in router and IoT web interfaces.

- **Cross-site scripting**

XSS. An attacker gets their own JavaScript into a web page that other users see, for example in a comment. The script runs in the victim's browser as if it came from the site, so it can steal session cookies or act as the user. It's prevented by escaping output and using a Content Security Policy.

- **Information disclosure**

A system reveals information it shouldn't. Examples are detailed error messages with stack traces, software version numbers in headers, exposed `.git` folders or backup files, and API responses with too much data. It's often not dangerous on its own, but it helps attackers plan the next step.

- **Code deobfuscation / obfuscation**

Obfuscation makes code deliberately hard to read without changing what it does, for example by renaming everything, encrypting strings or adding junk code. It's used to protect apps from copying and by malware to hide from analysis. **Deobfuscation** is the reverse: analysts try to make the code readable again.

- **OSINT**

Open-Source Intelligence. Collecting information from public sources like websites, social media, DNS records, job ads, GitHub and Shodan. Attackers use it to map a target before an attack, and defenders use it to see what they're exposing.

- **Data exfiltration**

Getting stolen data out of the victim's network without being noticed, for example by uploading it to cloud storage, hiding it in DNS queries, or sending it slowly in small pieces.

- **Lateral movement**

After getting into one machine, the attacker moves on to other machines in the same network, using stolen passwords, shared admin accounts or unpatched internal services, to reach more valuable systems like servers or the domain controller.

- **Command & Control**

C2 or C&C. The server an attacker uses to control compromised machines (bots) after the break-in: it sends them commands and receives stolen data. C2 traffic is often hidden inside normal-looking HTTPS or DNS traffic. The Mirai IoT botnet is a well-known example.

- **Social engineering**

Attacking people instead of technology: tricking someone into giving a password, clicking a link or doing something they shouldn't. Phishing emails, fake IT-support calls and USB sticks left in a parking lot are typical examples.

- **IDS/NIDS**

An Intrusion Detection System watches for signs of attacks and raises alerts. A **NIDS** (Network IDS) does this by monitoring network traffic, while a host-based IDS (HIDS) watches a single machine's logs and files. Detection uses known attack signatures and unusual behaviour. Examples: Snort, Suricata, Zeek. An IPS can also block the traffic, not just alert.

- **SIEM**

Security Information and Event Management. A system that collects logs from servers, firewalls, IDSs and applications in one place, connects related events together, and alerts the security team when something suspicious happens. Examples: Splunk, Microsoft Sentinel, Wazuh, Elastic Security.

59. Explain Microsoft's STRIDE threat model shortly (see the [old software vulnerability slides](#))

STRIDE is a threat modelling method from Microsoft. When you design a system, you go through each part of it (a device, an API, a data flow) and ask which of six types of threat could hit it. The name comes from the first letters, and each threat breaks one security property:

Threat	What it means	Property it breaks	IoT example
Spoofing	Pretending to be someone or something else, e.g. with stolen credentials or a fake IP address	Authentication	A fake sensor sends readings to the broker using a copied client ID
Tampering	Changing data without permission, e.g. while it travels over the network	Integrity	Someone modifies MQTT messages to change a thermostat's setpoint
Repudiation	A user can deny having done something, because there are no proper logs to prove it	Non-repudiation	No audit log shows who unlocked a smart door
Information disclosure	Private data is exposed to people who shouldn't see it, e.g. plaintext traffic or error messages	Confidentiality	Sensor data and passwords sent over unencrypted MQTT (port 1883)
Denial of service	Making a system unavailable, e.g. by flooding it with requests or crashing it with bad input	Availability	Flooding the broker or jamming the radio so the devices can't report
Elevation of privilege	A user with limited rights gets higher rights, like admin or root	Authorization	A command injection in a router's web UI gives the attacker root

The point is to find threats systematically during design, instead of relying on someone happening to think of them, and then plan a countermeasure for each one. For example: authentication against spoofing, signatures or TLS against tampering, logging against repudiation, encryption against disclosure, rate limiting against DoS, and least privilege against elevation.

60. Explain Microsoft's DREAD risk model shortly (see the [old software vulnerability slides](#))

STRIDE tells you *what kind* of threats exist. DREAD helps you decide *how serious* each one is, so you know which to fix first. Simply rating threats "high" or "low" leads to arguments, because everyone has a different gut feeling. So DREAD splits the rating into five questions:

- Damage potential: how bad is the damage if the vulnerability is exploited?
- Reproducibility: how easy is it to repeat the attack? Does it work every time, or only in rare conditions?
- Exploitability: how much skill, effort or access does the attack need?
- Affected users: roughly what share of users would be affected?
- Discoverability: how easy is it for an attacker to find the vulnerability?

Each question gets a score. In Microsoft's original guide it's 1 = low, 2 = medium, 3 = high, and many people use 0–10 instead. The scores are added together (or averaged), and the total gives the risk level. With the 1–3 scale, the total is between 5 and 15: 12–15 is high risk, 8–11 medium, and 5–7 low.

Example: an IoT device that sends its password over plain MQTT would score high on almost everything. An attacker who reads it gets full control of the device (damage), it works every time (reproducibility), only Wireshark is needed (exploitability), every device of that model is affected (affected users), and it's easy to spot (discoverability).

The weakness is that the scores are still subjective. Discoverability in particular is a problem, because it rewards hiding a bug instead of fixing it. Microsoft itself stopped using DREAD internally, and today CVSS is the more common way to rate vulnerabilities.

61. Check some CVEs of widely used applications from <https://www.cvedetails.com/> and answer:

- Describe what is the CVE scoring system

CVE itself doesn't score anything. It only gives each vulnerability an ID. The scores you see on [cvedetails.com](https://www.cvedetails.com/) come from **CVSS** (Common Vulnerability Scoring System), mostly calculated by NIST's National Vulnerability Database (NVD). The CVSS v3.1 base score is calculated from eight metrics:

- **Exploitability:** Attack Vector (network, adjacent, local, physical), Attack Complexity, Privileges Required, User Interaction
- **Scope:** whether the attack can affect other components beyond the vulnerable one
- **Impact:** effect on Confidentiality, Integrity and Availability

The result is a score from 0.0 to 10.0, plus a vector string like

`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H` (= 9.8) that shows how it was calculated.

Score	Severity
0.0	None
0.1–3.9	Low
4.0–6.9	Medium
7.0–8.9	High
9.0–10.0	Critical

A newer version, CVSS 4.0, came out in 2023. [cvedetails.com](https://www.cvedetails.com/) also shows an **EPSS** score, which estimates how likely a vulnerability is to be exploited in the wild in the next 30 days.

- When was the last time when Exim (MTA, mail transfer agent, more modern version of the application, not the Cambridge version) had a critical vulnerability? What is the CVE number?

CVE-2026-45185, published on **12 May 2026**, with a CVSS score of **9.8 (Critical)**. It's a use-after-free bug ("Dead.Letter") in how Exim handles BDAT (chunked) message bodies in builds that use GnuTLS, when the TLS connection is being shut down. An attacker who isn't logged in can send a crafted SMTP session that corrupts heap memory, which can lead to remote code execution. It affects Exim 4.97 up to 4.99.2 with STARTTLS and CHUNKING enabled, and was fixed in **Exim 4.99.3**.

Several Exim vulnerabilities have been published since then (between May and September 2026), but none of them is rated critical. The highest is CVE-2026-66140 at 8.4.

- Describe CVE-2016-6210 vulnerability shortly. Optional: How can you prevent such attack / vulnerability?

A **user enumeration** vulnerability in **OpenSSH before 7.3**, found with a timing attack. When someone tries to log in with a password, sshd hashes it. For a username that doesn't exist, it hashed a fixed fake password with the fast Blowfish algorithm. For a real user, it used the user's actual hash type (SHA-256/SHA-512 crypt), which is slower. If the attacker sends a very long password, for example 10 KB, the difference becomes clearly measurable: the server answers noticeably later when the user **exists**. This way an attacker can find out valid usernames and then focus password guessing on them.

Prevention:

- Update OpenSSH to 7.3 or newer, where the dummy hash uses the same algorithm as real users.
 - Disable password login (`PasswordAuthentication no`) and use SSH keys only. Then there's no password hashing to time.
 - Limit login attempts with `MaxAuthTries` , fail2ban or firewall rate limiting, and don't expose SSH to the whole internet if it's not needed (use a VPN or an IP allowlist).
 - In general, use the same code path and constant-time operations for valid and invalid users (see CWE-208 below).
- Describe CVE-2019-15846 vulnerability shortly

A critical **remote code execution as root** in **Exim before 4.92.2** (CVSS 9.8), published in September 2019. During the TLS handshake, the client sends the server name (SNI). If the SNI ends with a backslash followed by a null byte (`\ + \0`), Exim's string unescaping function reads and writes past the end of the buffer, causing a heap overflow. An attacker without a login could use this to run code as root. Since Exim was used on more than half of all internet mail servers at the time, millions of servers were exposed. It was fixed in Exim 4.92.2.

- Describe CWE-208 from <https://cwe.mitre.org/data/archive.html> (download most recent PDF)

CWE-208: Observable Timing Discrepancy. Two operations take a measurably different amount of time depending on something secret, so an attacker can learn security-relevant information just by timing the responses. It's a child of CWE-203 (Observable Discrepancy).

Typical examples:

- Login takes longer for an existing user than a non-existing one. This is exactly CVE-2016-6210.
- A password or token is compared byte by byte and the comparison stops at the first wrong byte. The attacker can guess the secret one character at a time, because each correct character makes the check slightly slower.
- Cryptographic code whose timing depends on the key, which lets attackers recover the key (a side-channel attack, see question 63).

It's prevented with **constant-time** code: compare secrets with constant-time functions (e.g.

`hmac.compare_digest` in Python, `CRYPTO_memcmp` in OpenSSL), run the same code path for valid and invalid input, and use well-tested crypto libraries instead of your own implementation.

62. Study [D-Link DNS-320 ShareCenter write-up](#) in the ExploitDB

- What kind of software exploit is that?

A remote, unauthenticated backdoor exploit against a NAS device. The write-up is about the **DNS-320L** model, firmware older than 1.06, and says other ShareCenter devices may be affected too. It was published by James Bercegay / GulfTech in January 2018. It combines two vulnerabilities:

- **Hard-coded credentials (backdoor):** the firmware has a secret built-in username and password that work on every device and can't be changed by the owner.
 - **Command injection:** user input is passed to shell commands without filtering. Together they give remote code execution as root.
- Try to explain shortly (summarise) from the write-up, how the attacker can elevate access to become root (administrator) user?
 1. The CGI program `nas_sharing.cgi` contains a hard-coded login, user `mydlinkBRionyg` with password `abc12345cba` (sent base64-encoded as `YWJjMTIzNDVjYmE`). With these, the attacker gets admin access on any affected device without knowing the owner's password.
 2. In the same program, the `cmd=15` function passes a parameter straight into a shell command, so the attacker can run any command they want.
 3. The attacker turns this into a permanent web shell. First, they "log out" via `login_mgr.cgi?cmd=logout` with PHP code as the username. The name is written to a log file without filtering. Then they use the `cmd=15` injection to copy that content into `/var/www/shell.php`.
 4. Now `http://<nas>/shell.php` runs any PHP or system command they send **as root**, so the attacker has full control of the NAS and all the files stored on it.

No user interaction or real account is needed. Even the logout request works without being logged in. So any affected device reachable from the network (or internet) could be taken over.

63. Read this short [article about cracking SIM cards](#) and answer these questions:

- What is “side-channel attack”?

An attack that doesn't break the algorithm itself, but uses **physical information that leaks while the device is computing**. Examples are power consumption, electromagnetic emissions, heat, sound or timing. Because the chip's power use depends slightly on the data and key it's processing, measuring it many times and analysing the results statistically can reveal the secret key. The maths of the encryption can be perfectly secure, and the implementation still leaks the key.

- How side-channel attack was used to crack SIM cards?

At Black Hat 2015, researchers led by Yu Yu from Shanghai Jiao Tong University showed that the **AES-128 key** in 3G/4G USIM cards can be extracted with **Differential Power Analysis (DPA)**:

- They connected the SIM to a card reader and PC, and measured its **power consumption with an oscilloscope** while it ran the authentication algorithm with many different inputs. A protocol analyzer (MP300-SC2) recorded the traffic.
- By correlating the power traces with the inputs, they worked out the secret key a piece at a time.
- Cracking each of eight commercial SIM cards took only **10 to 80 minutes**.

With the key, they could **clone the SIM card**, pretend to be the owner on the mobile network and, for example, take over accounts that use SMS verification, like Alipay. The cards were vulnerable because they ran a plain software AES with no side-channel countermeasures, such as masking or adding random noise or delays.

64. Browse this [public penetration test report](#) and [news article](#) and answer these questions:

Background: in 2016, security firm MedSec and investment firm Muddy Waters published vulnerabilities in St. Jude Medical's implantable heart devices (pacemakers and defibrillators, ICDs) and the **Merlin@home** bedside transmitter that sends their data to doctors. St. Jude sued, and Bishop Fox was hired as an independent expert to check the claims. This report is that expert's report, and it confirmed the attacks work.

- Penetration test report has header *security through obscurity* (next to the item 171 and onwards). What does it mean?

Security through obscurity means relying on keeping the design secret (hidden protocols, undocumented hardware, "nobody knows how it works") as the main protection, instead of real security like authentication and encryption. It's considered bad practice, because secrets like these are eventually found through reverse engineering, and then there's nothing left protecting the system. A system should stay secure even if the attacker knows exactly how it works (Kerckhoffs's principle).

In the report, St. Jude claimed MedSec was supporting security through obscurity. MedSec had pointed out that St. Jude's use of off-the-shelf parts and the lack of anti-debugging protections made Merlin@home easy to reverse engineer. The expert disagreed (items 174–175). Well-documented standard chips with public datasheets and no anti-debugging protection simply *are* easier to analyse, and saying so is just a practical observation from reverse engineering, not an argument that hiding things is real security. The actual problems were the missing protections, like leftover debug scripts and no authentication.

- Penetration test report items 114 - 141 describe remote attack and vulnerability. What kind of problem is it?

A **battery drain attack**, a kind of **denial-of-service** attack against a medical implant. A rooted Merlin@home transmitter ran a script that repeatedly woke the implanted defibrillator over radio (2.4 GHz wake-up, then a 400 MHz session), interrogated it and disconnected, in a continuous loop. The implant **didn't authenticate** who was talking to it, so it answered every time and used battery power doing so.

Bishop Fox reproduced it:

- The target ICD lost about **3% of its battery per 24 hours**, while a control device shielded in Faraday bags lost nothing. Running non-stop, that would empty the battery in about **33 days**, or about three months if only run while the patient sleeps next to the transmitter.
- The attack also worked from **10 feet (3 m)** away, with the ICD wrapped in bacon and minced meat to simulate a human body.

St. Jude said the patient gets a low-battery alert. The expert disagreed that there is "no credible threat", because the attack is reliable, repeatable and works under real-world conditions. An implant that a patient's life depends on could stop working much earlier than expected.

65. Read this news article about [garage door security vulnerability](#) and answer:

The article is about **Nexx** smart garage door controllers, smart plugs and alarms. Security researcher **Sam Sabetan** found the problems, and there were an estimated 40,000+ devices linked to about 20,000 accounts.

- What information security and privacy issues were found and listed in the article?
 - **Anyone could open any customer's garage door** anywhere in the world, and switch Nexx smart plugs on or off, without any access to the owner's account.
 - **Privacy leak:** the attacker could see all customers' MQTT traffic, including **email addresses, names, device IDs** and the **open/close events** of garage doors. That reveals where people live and when they come and go, which is useful for burglars.
 - **API flaws (IDOR):** with just a device ID, it was possible to read another user's device history and information and change device settings.
 - Devices could be **re-registered by anyone who knew the MAC address**, which also leaked the owner's data.
 - **The vendor never responded.** Sabetan and CISA tried to contact Nexx, but the flaws were left unpatched, so CISA published an advisory and owners were told to unplug the devices.

- What was the main issue and vulnerability with MQTT configuration/architecture?

The Nexx cloud gave **every device the same universal MQTT username and password**, and that password could be found in the device firmware and in the mobile app's API traffic. All devices and apps connected to **one shared cloud broker**, and the broker had **no per-device access control (ACLs)**. So anyone logged in with the shared credentials could:

- subscribe to all topics (e.g. with the **#** wildcard), seeing every customer's messages, and
- publish to any device's topic, e.g. **replay a recorded "open" command** to open someone else's garage.

A correct design would give each device its own credentials or client certificate, and broker ACLs that only allow a device or user to access its own topics.

- Read the **CVE-2023-1748** (it's about this vulnerability). How much (i.e. how bad) is the base CVSS score? What is the CWE code for this kind of vulnerability?

The base score is **CVSS v3.1 9.3 (Critical)** from CISA (`AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:L`), and NVD's own rating is even **10.0**. The CWE is **CWE-798: Use of Hard-coded Credentials**.

66. Browse this “Secure development - towards approval” [PDF document](#) from National Cyber Security Centre Finland and answer from TESTING AND VERIFICATION chapter:

- What is unit testing?

Automated tests written and run by the developers themselves, testing small pieces of code (functions, classes) inside one component. Since developers run them all the time, bugs are found early, when they're fast and cheap to fix. Code reviews are a good place to check that unit tests exist.

- What is component testing?

Testing one component **in isolation**, with the rest of the system replaced by simulated components (mocks). These tests may be designed by a separate test team, and they should ideally be automated and run every day or night.

- What is system testing?

Testing a build of the **whole system** together. It often needs manual work, so it's slower and more expensive than unit or component tests. It can be automated, but that may need a big investment in test infrastructure.

- What is acceptance testing?

Testing done by an **independent party**, such as a separate test team, the customer or a third party, to decide whether the system is accepted. Problems found this late can be very expensive and slow to fix, because they can mean big changes and repeating the acceptance tests.

- What is static testing?

Testing **without running** the product, by inspecting things like source code and binaries. Code reviews and inspections, and automated source code analysis, are static testing. So is **software composition analysis**, which inspects a compiled binary to find out which third-party components it's built from.

- What is dynamic testing?

Testing by **running** the product and observing how it behaves, either manually or automatically. Traditionally this checks that the product meets its requirements. Today it should also test the security requirements and try to attack and abuse the product. Fuzzing and load testing (performance) are forms of dynamic testing.

- What is fuzzing?

A security-focused dynamic testing method where the product is fed **unexpected, malformed and random inputs** to find bugs. The first sign of a vulnerability is usually a crash or the product stopping responding, and with specially crafted input an attacker might even take control. Fuzzing doesn't need the source code and can be fully automated, which is also why attackers use it. The guide's advice is to fuzz your product yourself so you find the problems before others do, especially your own custom interfaces and protocols.

67. Browse this “Instructions – Supply chain attack” [PDF document](#) from National Cyber Security Centre Finland and research/answer:

- What is supply chain attack?

An attack where the attacker breaks into an organisation **through something it trusts**: a supplier, service provider, software product, update channel or open-source component. They first compromise the supplier and plant malicious code in its product. The infected product then reaches all the customers through the normal distribution channel, for example as a signed software update. Because the code comes from a trusted source, it's hard to detect, and one break-in can give the attacker a foothold in hundreds or thousands of organisations at once. From there it's used for further attacks like data theft or ransomware.

Well-known examples are **SolarWinds Orion** (2020, a backdoored update installed by around 18,000 organisations), **Kaseya VSA** (2021, ransomware pushed through an IT management tool) and the **xz Utils backdoor** (2024, planted by a fake open-source maintainer and caught just before it spread widely).

- What is 3-2-1 backup rule?

Keep at least **3 copies** of your data (the original plus two backups), on **2 different types of media or formats**, with **1 copy offline or off-site**, completely outside the network. The offline copy is the important part against ransomware and supply chain attacks, because an attacker who controls the network can't encrypt or delete it. The guide also stresses **testing the backups and practising restoring** them, because a backup you can't restore doesn't help.

- What is network segmentation and how/why it improves information security?

Dividing a network into **separate zones**, using VLANs, subnets and firewalls between them, and only allowing the traffic each zone really needs. For example, office computers, servers, guest Wi-Fi, management interfaces and IoT devices each get their own segment.

It improves security because:

- **It limits lateral movement**: if an attacker or malware gets into one device, for example a hacked IoT camera or an infected supplier tool, it can't directly reach the rest of the network. The damage stays in that segment.
- **It makes the attack surface smaller**: critical systems are only reachable from the segments that actually need them.
- **It makes detection easier**: traffic crossing between segments goes through firewalls, where it can be logged and monitored, and unusual connections stand out.
- **It makes isolation faster during an incident**: an infected segment can be cut off without shutting down everything.

For IoT this is especially important, because the devices are often weakly secured and rarely updated. Putting them in their own VLAN keeps a compromised smart device away from computers and servers.

68. Check some recent vulnerabilities being exploited in the wild from cisa.gov. Select one, summarise the problem, and search and study some news articles about the vulnerability

I picked the "**MikroTrick**" exploit chain in **MikroTik RouterOS**, because MikroTik routers are everywhere in small businesses, ISPs and IoT and industrial networks. Routers like these are a favourite target for botnets.

Item	Details
CVEs	CVE-2026-67279 (SSH authentication bypass) + CVE-2026-86060 (privilege escalation)
Added to CISA KEV	CVE-2026-86060 on 10 Sep 2026, CVE-2026-67279 on 25 Sep 2026
CWE	CWE-841 (Improper Enforcement of Behavioral Workflow), CWE-88 (Argument Injection)
CVSS	6.5 (Medium) for CVE-2026-67279 on its own, but the chain gives full admin control without a password
Found by	Sławomir Rozbicki, CERT Polska
Fixed in	RouterOS 6.49.21, 7.23.4 and 7.24.2 (released 3 Sep 2026)

The problem

1. **CVE-2026-67279**: An SSH connection normally goes in a fixed order: key exchange, then **user authentication**, then opening a session. RouterOS's SSH server could be tricked into skipping the authentication step. If the client asked for a **rekey** (a new key exchange) before logging in, the server afterwards behaved as if the login had already happened. It then accepted session channels and `exec` commands from a client that never gave a password. That's why the CWE is about not enforcing the correct order of steps (the workflow).
2. **CVE-2026-86060**: Usernames starting with a forbidden character were not handled properly when RouterOS passed them to its login process. An attacker could use this to change the **policy mask** that decides what rights the session has, and give themselves full admin rights.

Chained together, an attacker who can reach the router's SSH port gets a **full administrator console without any credentials**. CERT Polska also reported four other RouterOS bugs at the same time, for example CVE-2026-67276, where incomplete RSA key checking lets someone log in with a key without having its private part.

What the news says

- Attacks were seen **from 2 September 2026, one day before the patch** came out, so for a moment it was effectively a zero-day.
- After getting in, attackers created hidden admin accounts (often named `ops`) to keep access.
- Cybernews reported that around **122,500 MikroTik routers** with SSH reachable from the internet could be exposed. A proof-of-concept was published on GitHub soon after, which usually leads to mass scanning.
- The default RouterOS configuration blocks SSH from the internet. So the devices at risk are mostly ones where an admin opened SSH for remote management, which is common with ISPs and remote sites.

What to do

- Update RouterOS to 6.49.21 / 7.23.4 / 7.24.2 or newer.
- Don't expose SSH (or any management interface) to the internet. Allow it only from a VPN or a list of trusted IP addresses.
- Check for unknown user accounts and configuration changes. Patched versions also show a "Flagged" warning if they detect signs of compromise.
- If a router was compromised, **factory reset** it and configure it again by hand. Don't restore an old backup, because it may contain the attacker's changes.

This case connects to several earlier questions: an **authentication bypass** and **privilege escalation** (question 58), a network device used as an entry point for **lateral movement**, and a reason for **network segmentation** with management interfaces in their own protected segment (question 67).

Sources: [CISA KEV catalog](#), [CERT Polska advisory](#), [eSecurity Planet](#), [Cybernews](#), [Hexnode Threat Watch](#), [SentinelOne](#)